Objective: The purpose of this exercise is to learn how to connect simple input and output devices to an FPGA chip and implement a circuit that uses these devices. The problems in this lab exercise illustrate me to know about DE1-Soc board, Multiplexer, gates, switches, LED, 7 segment display.

- **Part1:** This is a basic program where we get output for corresponding switch. For example, when We turn on switch (0), it turns on the switch (1). The code screenshot and a sample output what I get are given below:

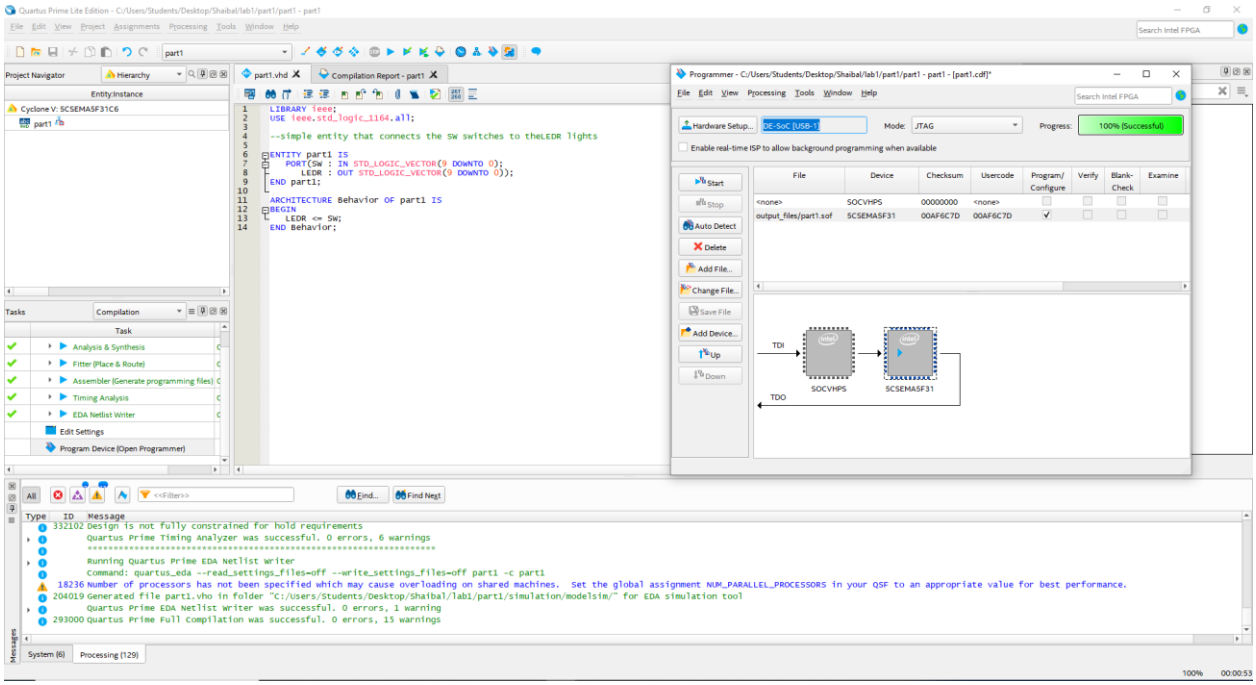


Figure 1.1: Code and analysis successful screenshot for part1

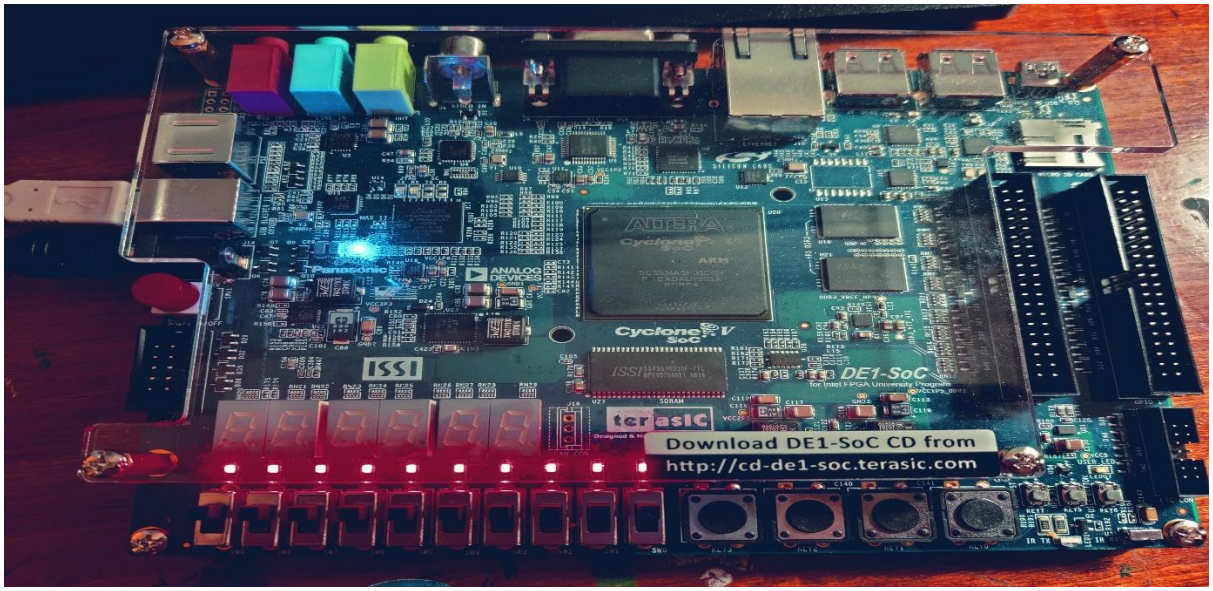


Figure1.2: Sample Output Picture of part1

- **Part2:** The part2 is to build a 4-bit wide 2 to 1 mux using 1 bit 2 to 1 mux. The truth table for 1 bit 2 to 1 is given below:

S0	output
0	x
1	y

Here for four bit we can think x as x0, x1, x2, x3 and Y as y0, y1, y2, y3. All the mux has same selection which makes this truth table as a generic table for models. For example, if you give x=0101 and y=1001 with selection s0=1 we get 1001 as output in this program. A clear picture of input of a sample attached below.

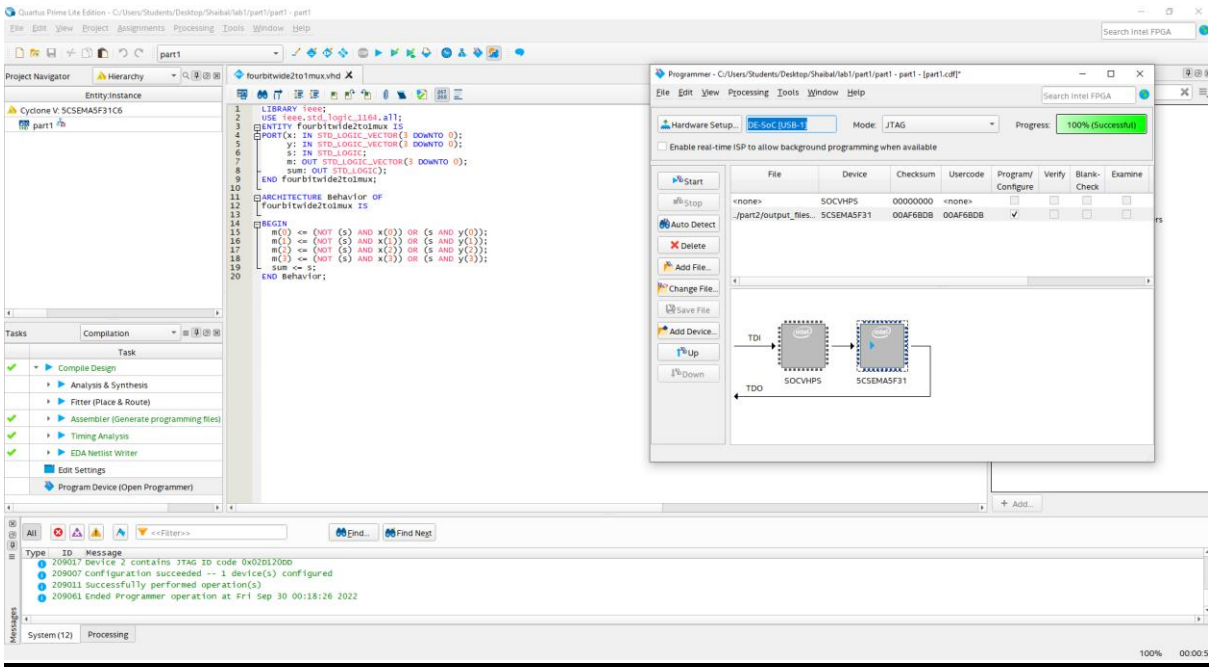


Figure 2.1: Code and analysis successful screenshot for part2

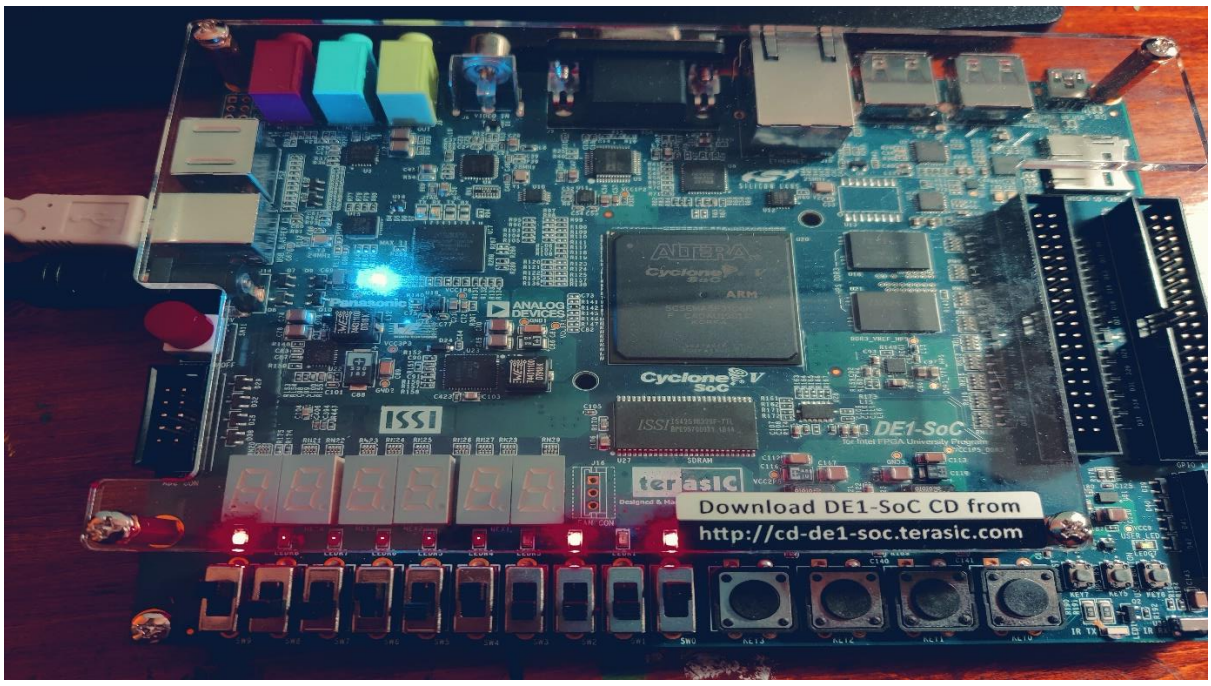
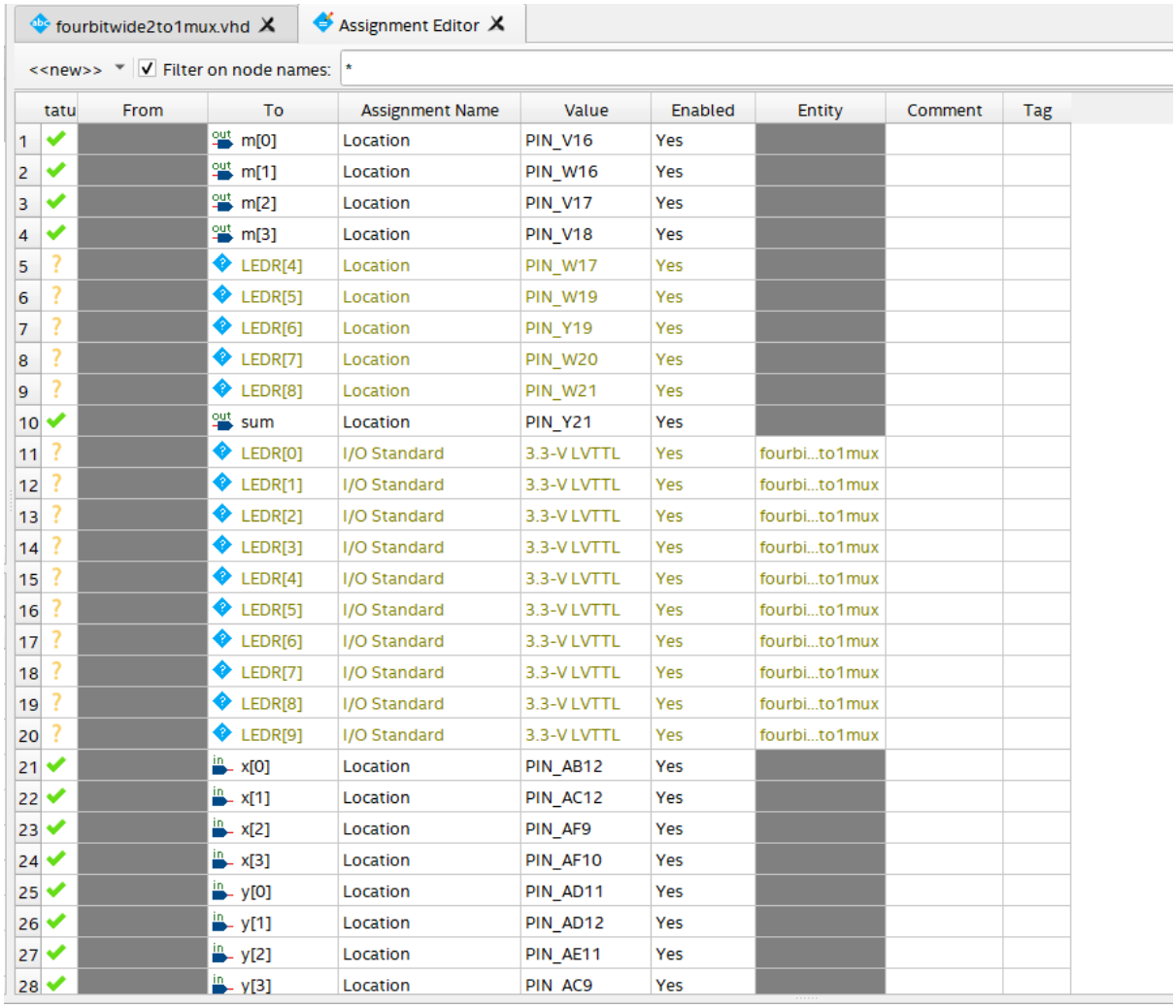


Figure 2.2: A sample output for s1=1



fourbitwide2to1mux.vhd | Assignment Editor

<<new>> | Filter on node names: *

	tatu	From	To	Assignment Name	Value	Enabled	Entity	Comment	Tag
1	✔		out m[0]	Location	PIN_V16	Yes			
2	✔		out m[1]	Location	PIN_W16	Yes			
3	✔		out m[2]	Location	PIN_V17	Yes			
4	✔		out m[3]	Location	PIN_V18	Yes			
5	?		LEDR[4]	Location	PIN_W17	Yes			
6	?		LEDR[5]	Location	PIN_W19	Yes			
7	?		LEDR[6]	Location	PIN_Y19	Yes			
8	?		LEDR[7]	Location	PIN_W20	Yes			
9	?		LEDR[8]	Location	PIN_W21	Yes			
10	✔		out sum	Location	PIN_Y21	Yes			
11	?		LEDR[0]	I/O Standard	3.3-V LVTTL	Yes	fourbi...to1mux		
12	?		LEDR[1]	I/O Standard	3.3-V LVTTL	Yes	fourbi...to1mux		
13	?		LEDR[2]	I/O Standard	3.3-V LVTTL	Yes	fourbi...to1mux		
14	?		LEDR[3]	I/O Standard	3.3-V LVTTL	Yes	fourbi...to1mux		
15	?		LEDR[4]	I/O Standard	3.3-V LVTTL	Yes	fourbi...to1mux		
16	?		LEDR[5]	I/O Standard	3.3-V LVTTL	Yes	fourbi...to1mux		
17	?		LEDR[6]	I/O Standard	3.3-V LVTTL	Yes	fourbi...to1mux		
18	?		LEDR[7]	I/O Standard	3.3-V LVTTL	Yes	fourbi...to1mux		
19	?		LEDR[8]	I/O Standard	3.3-V LVTTL	Yes	fourbi...to1mux		
20	?		LEDR[9]	I/O Standard	3.3-V LVTTL	Yes	fourbi...to1mux		
21	✔		in x[0]	Location	PIN_AB12	Yes			
22	✔		in x[1]	Location	PIN_AC12	Yes			
23	✔		in x[2]	Location	PIN_AF9	Yes			
24	✔		in x[3]	Location	PIN_AF10	Yes			
25	✔		in y[0]	Location	PIN_AD11	Yes			
26	✔		in y[1]	Location	PIN_AD12	Yes			
27	✔		in y[2]	Location	PIN_AE11	Yes			
28	✔		in y[3]	Location	PIN_AC9	Yes			

Figure 2.3: A pin configuration for part2 assignment

- **Part3:** The part3 is a problem to write VHDL program for a two-bit wide 4 to 1 mux. I used same subscript to make a 1 bit 4 to 1 mux to 2 bit 4 to 1 mux and show those output in led . The program. The truth table for 4 to 1 mux is below:

S1	S0	out
0	0	u
0	1	v
1	0	w
1	1	x

Here for two bit we can think u as u0, u1 and v as v0, v1 and w as w0,w1 and x as x0, x1. All the mux has same selection which makes this truth table as a generic table for models. For example, if you give u=01 ,v=10,w=00 and x=10 with selection s=00 we get 01 as output in this program. A clear picture of input of a sample attached below.

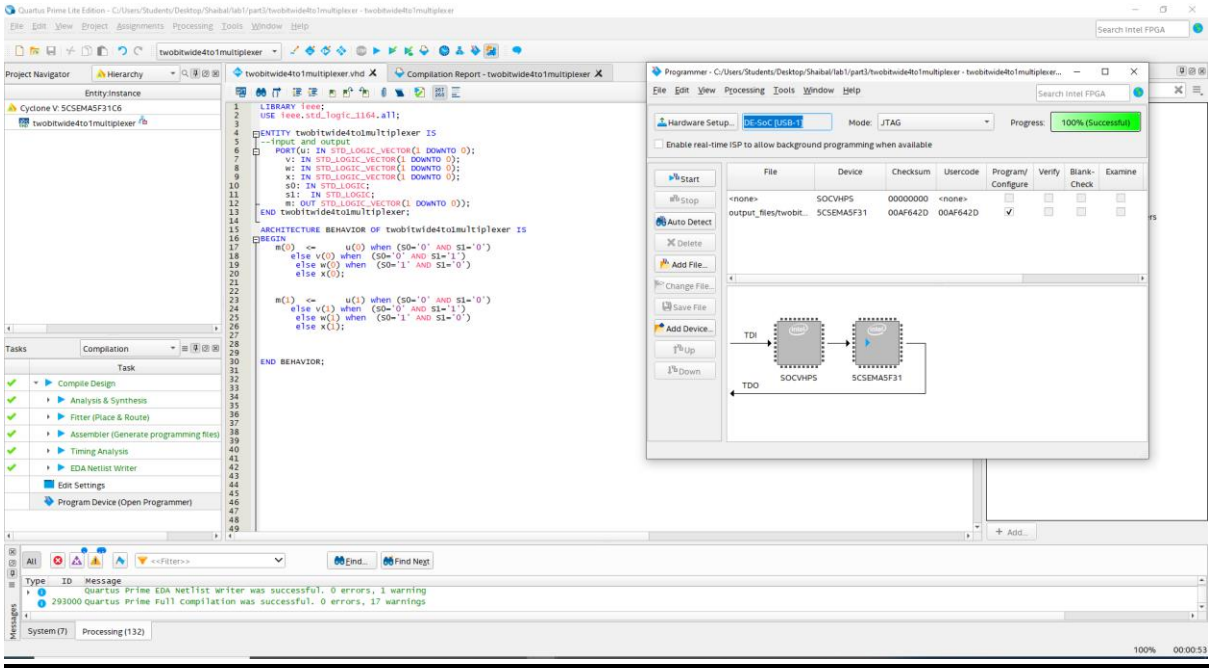


Figure 3.1: Code and analysis screenshot

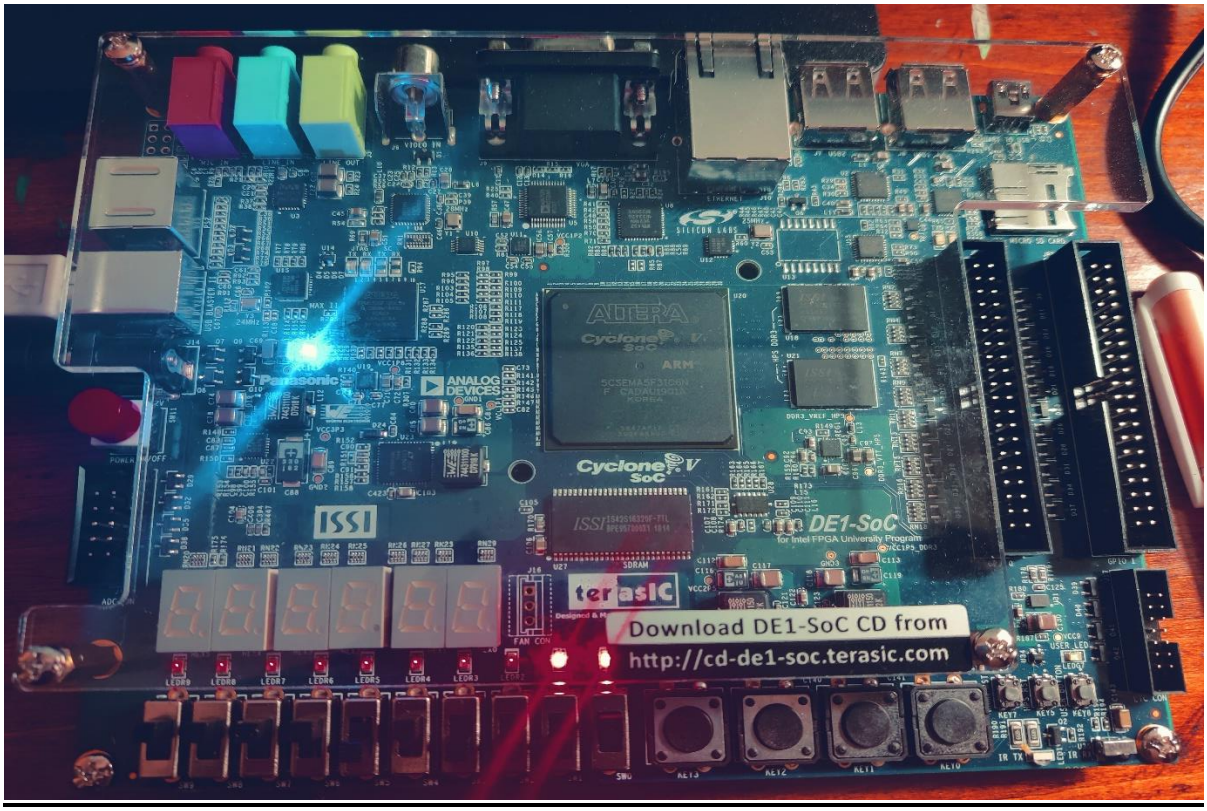


Figure 3.2: A sample picture of output

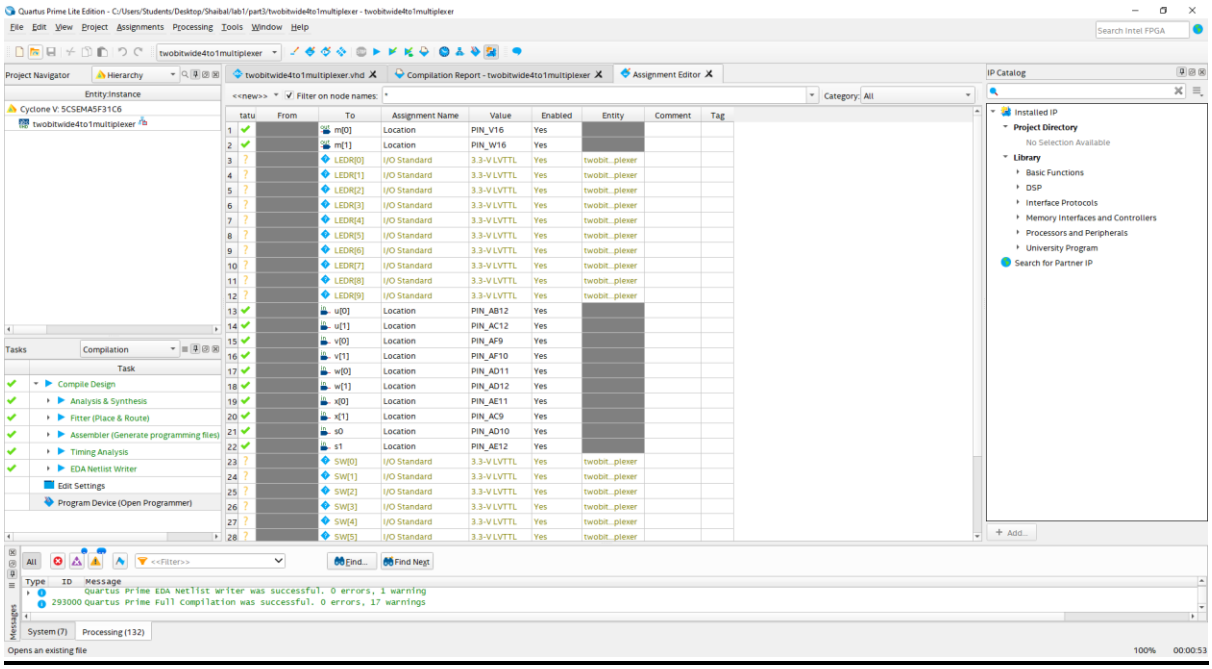


Figure 3.3: Pin configuration for part 3

- **Part4:** The part4 is a problem to write VHDL program for a 7 segment display which produces a specific letter with input.

c1	c0	output
0	0	d
0	1	E
1	0	1
1	1	blank

I use canonical approach of sop to find the appropriate equations. The assignments are for separate anode in 7 segment display. like I build equation for a,b,c,d,e,f,g nodes. The equations are given below: Here Hex0 is the 7-segment display and 0 to 6 are a to g respectively. The screenshots are given below step by step after equations:

HEX0(0)<= not(((not (c(1))) AND (c(0))));

HEX0(1)<= not(((not(c(1))) AND not(c(0))) OR (c(1) AND not(c(0))));

HEX0(2)<= not(((not(c(1))) AND not(c(0))) OR (c(1) AND not(c(0))));

HEX0(3)<= not(((not(c(1))) AND not(c(0))) OR (c(0) AND not(c(1))));

HEX0(4)<= not(((not(c(1))) AND not(c(0))) OR (c(0) AND not(c(1))));

HEX0(5)<= not(((not (c(1))) AND (c(0))));

HEX0(6)<= not(((not(c(1))) AND not(c(0))) OR (c(0) AND not(c(1))));

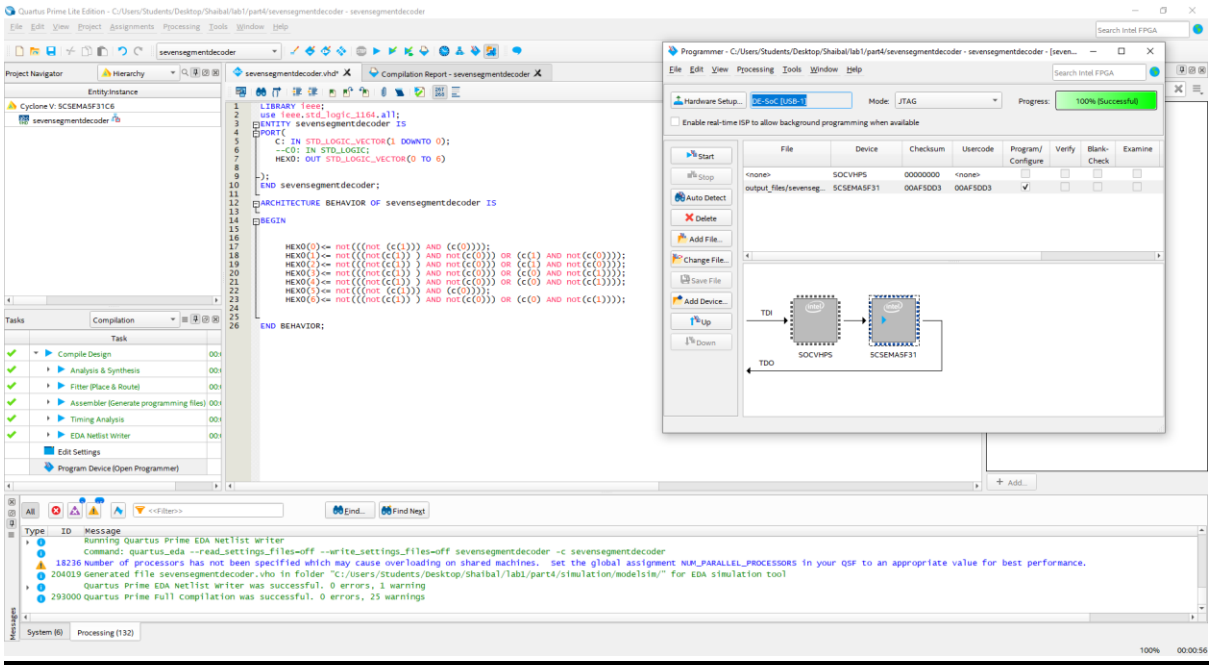


Figure 4.1: Code and analysis screenshot

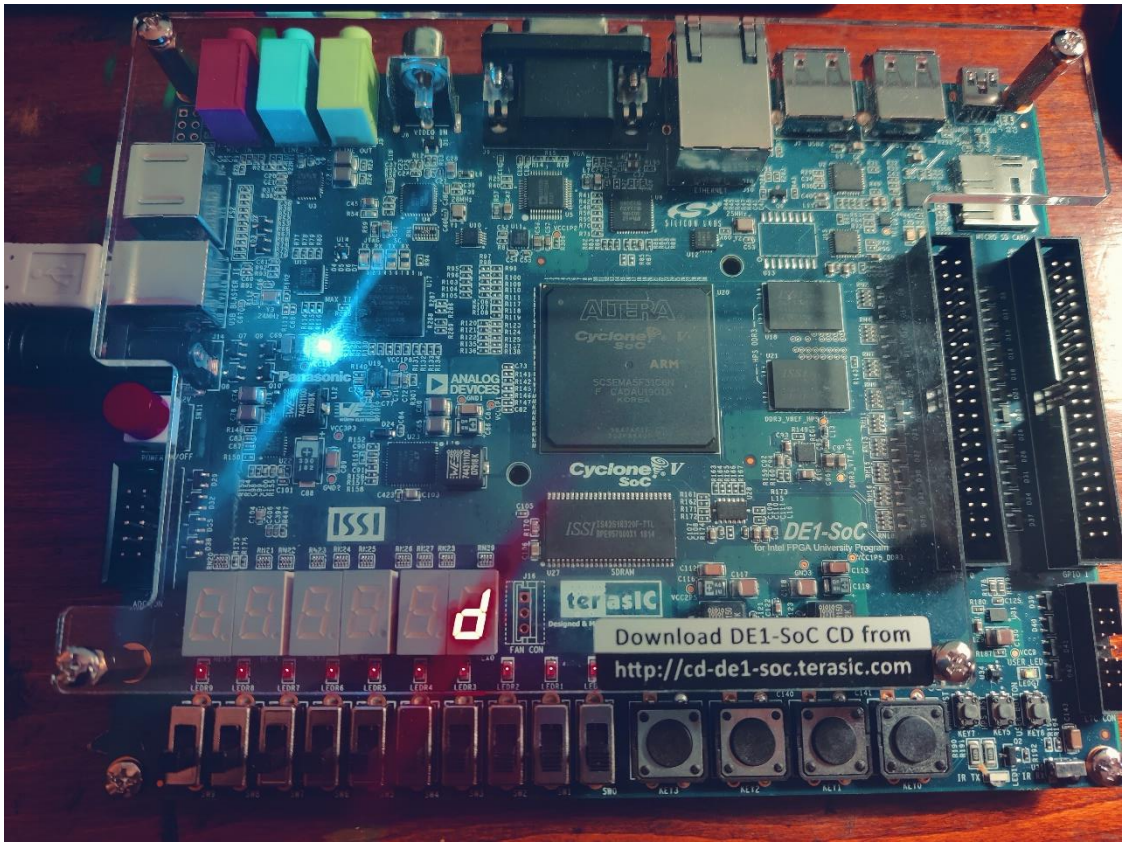


Figure 4.2: Output for 00

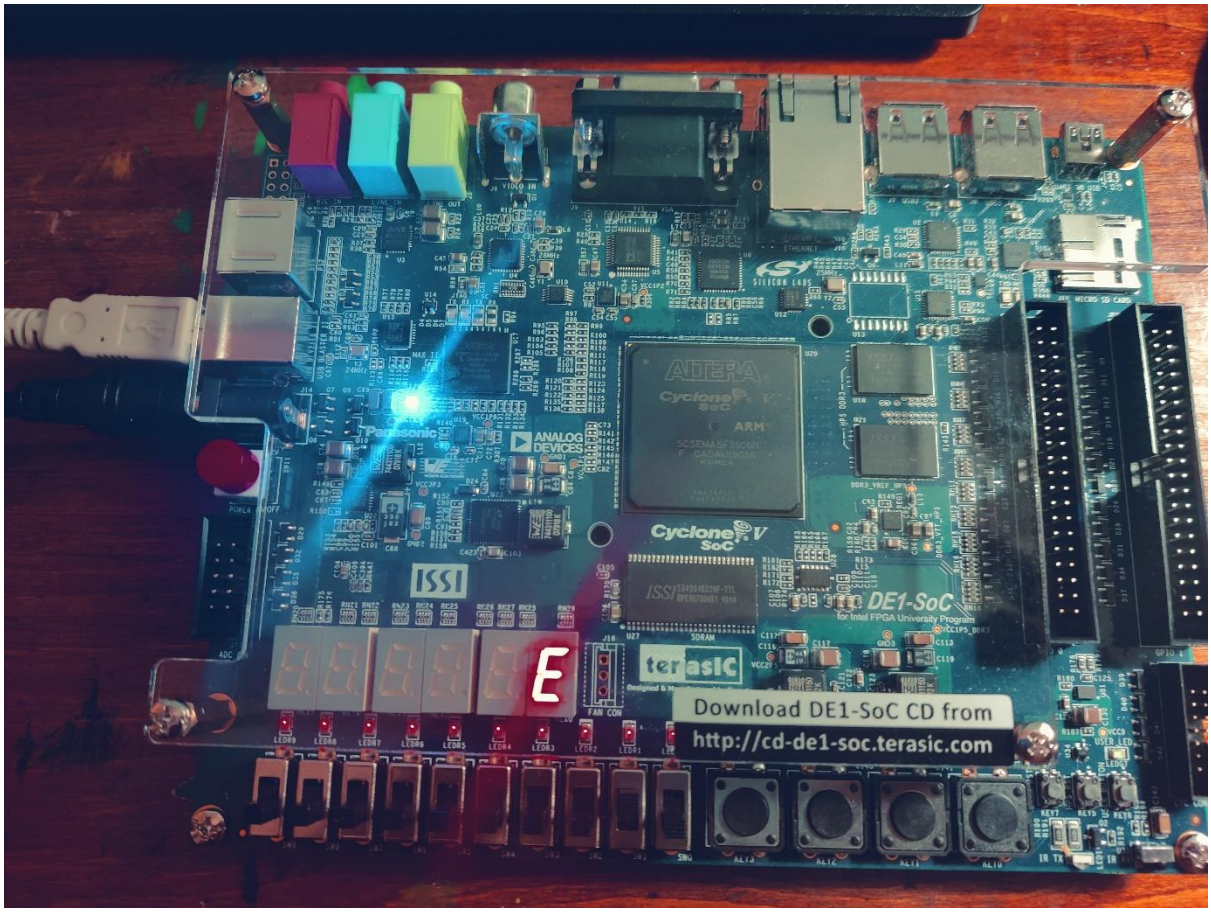


Figure 4.3: Output for 01

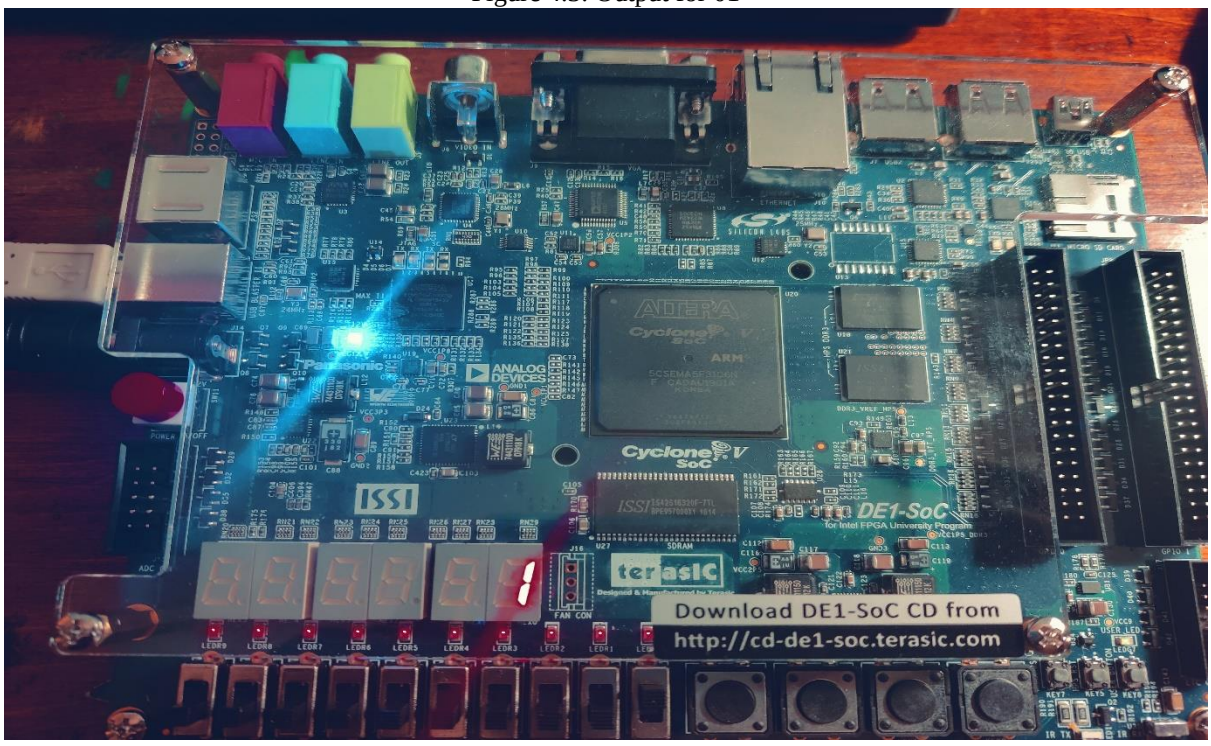


Figure 4.4: output for 10

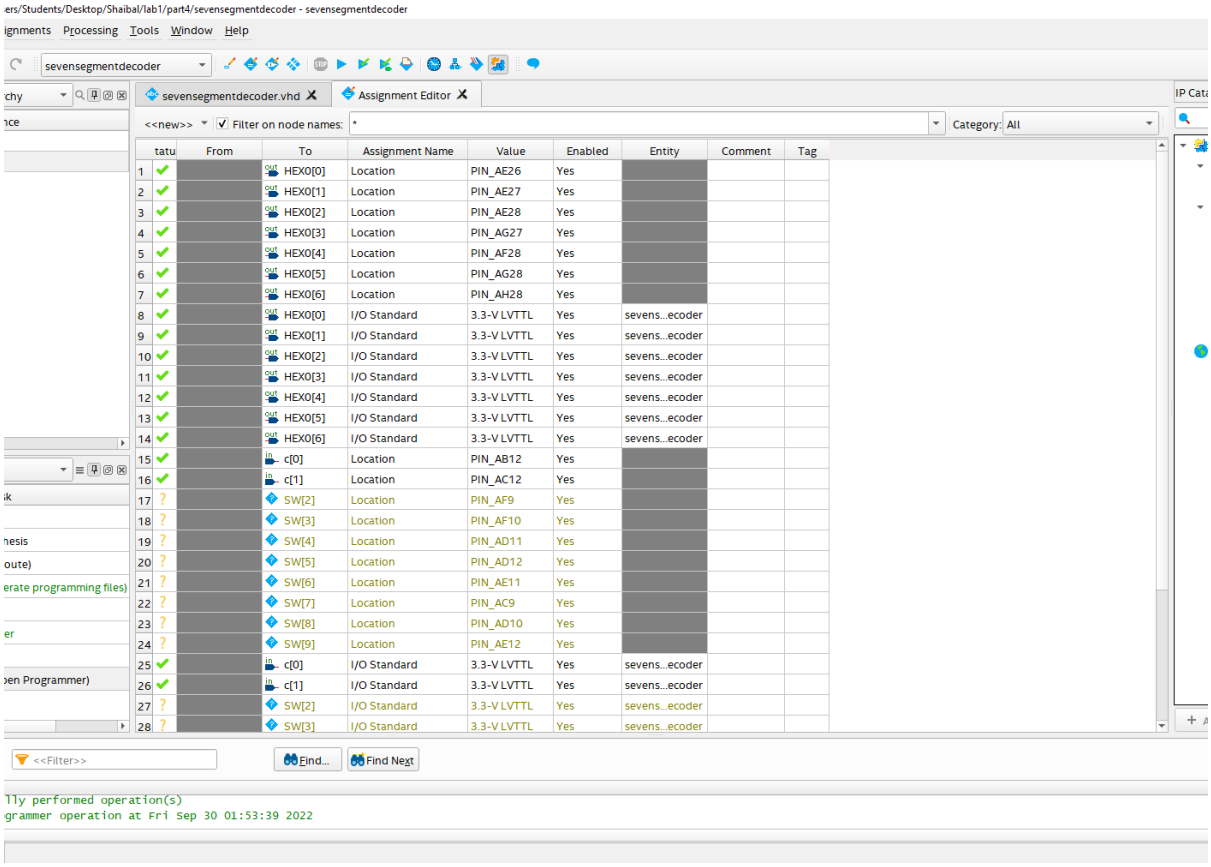


Figure 4.5: Pin assignment for part4

- **Part5:** The part5 is the combination part for part4 and part5. I have to use 7 segment concepts with mux 4 to1 mux concept where selector is responsible to print de10 word as pattern. In addition, I also did the rotation for the de10 as it is mentioned in the assignment documentation. I use component concept of VHDL programming to use the attribute of another file. I have to use four 7-segment display for this part. The detailed code and sample output along with pin assignment screenshots are attached below.

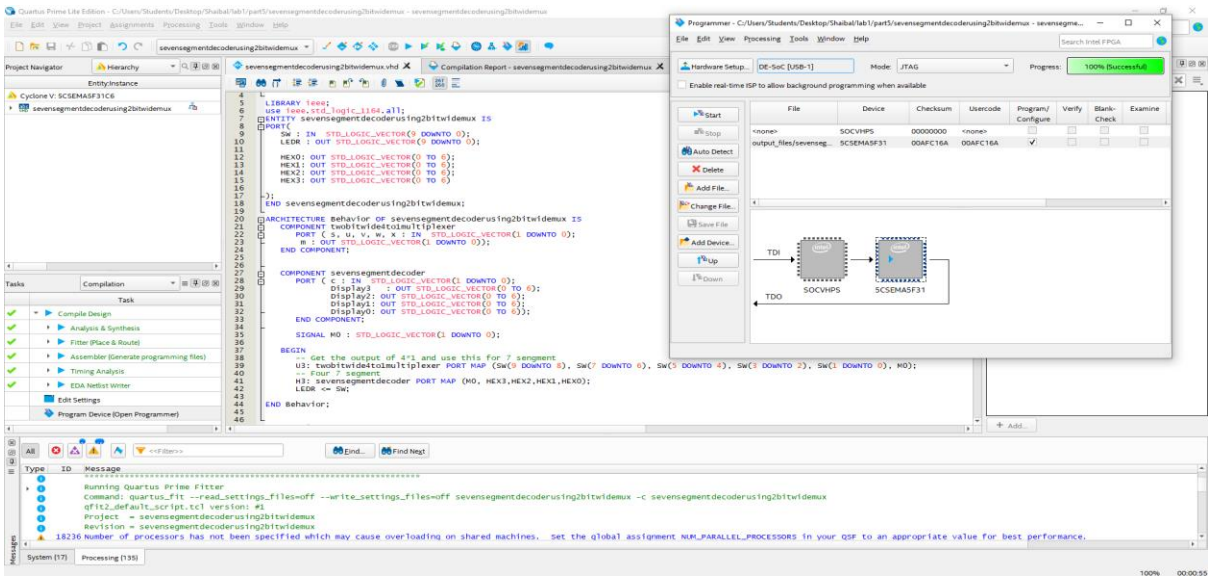


Figure 5.1: Code of part5

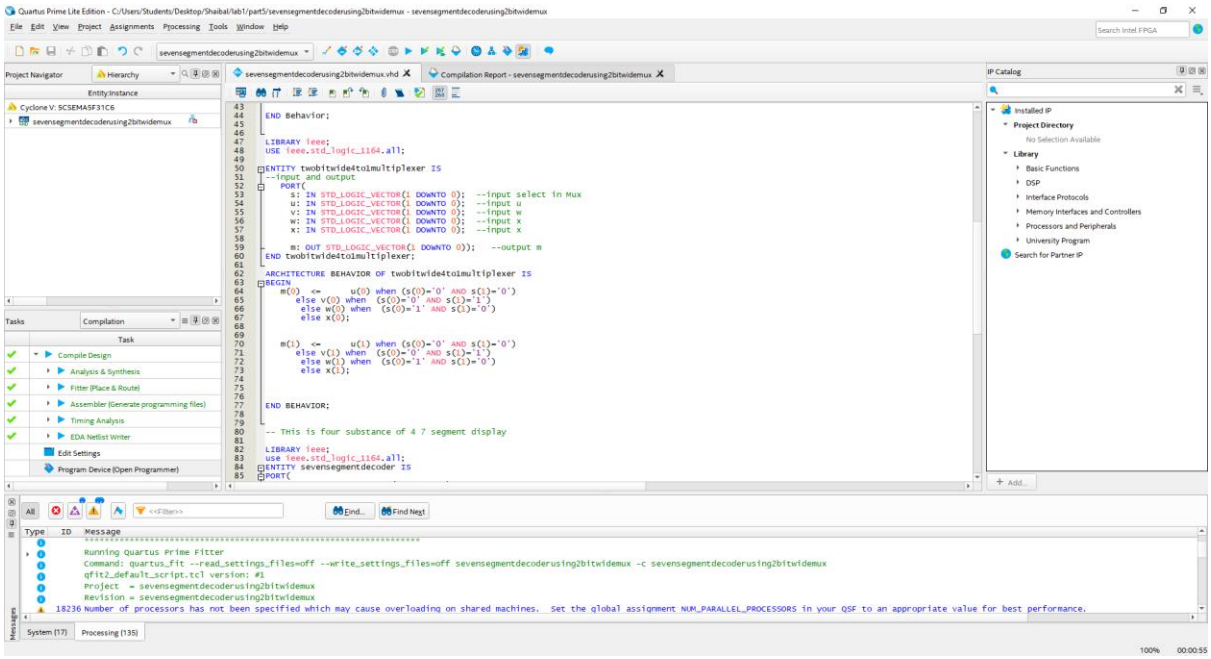


Figure 5.2: Code of part 5 (2bit 4 to1 mux)

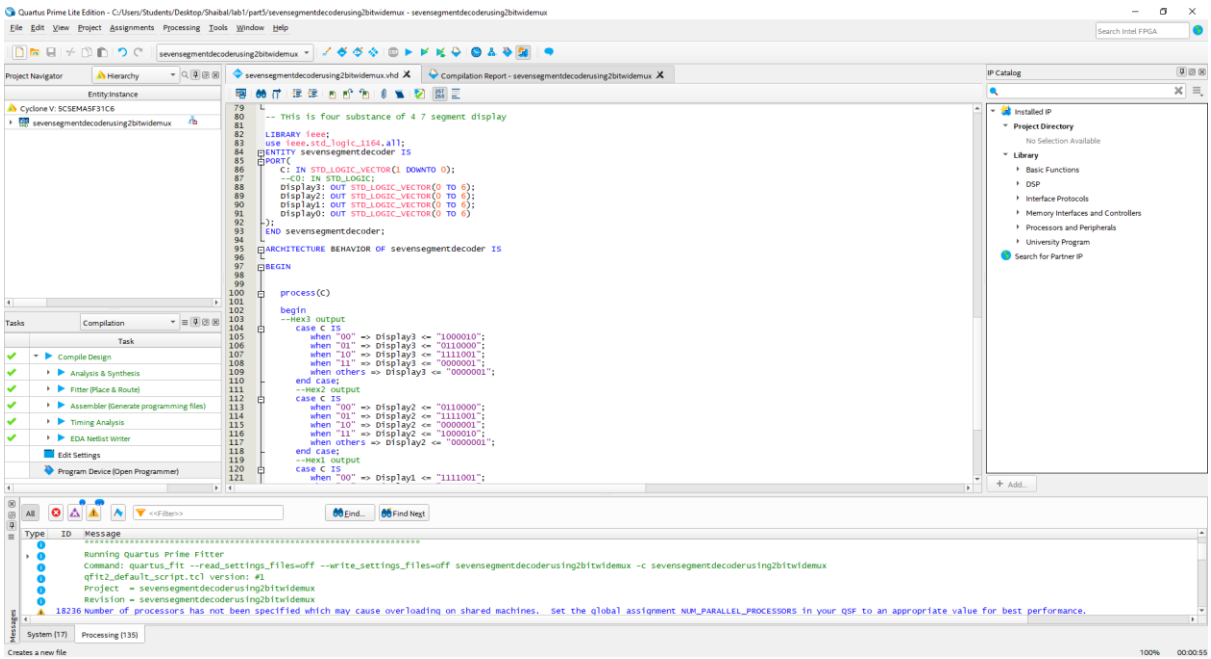


Figure 5.3: Code of part 5 (four 7-segment codes)

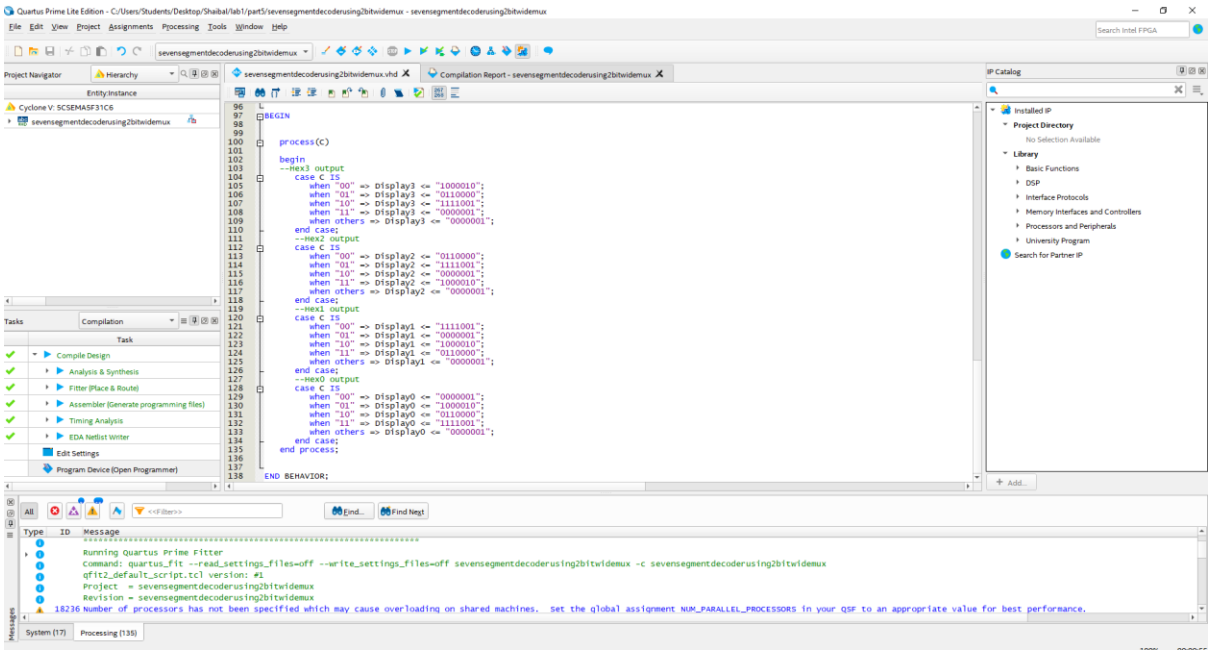


Figure 5.4: Code of part 5 (four 7-segment codes continue)

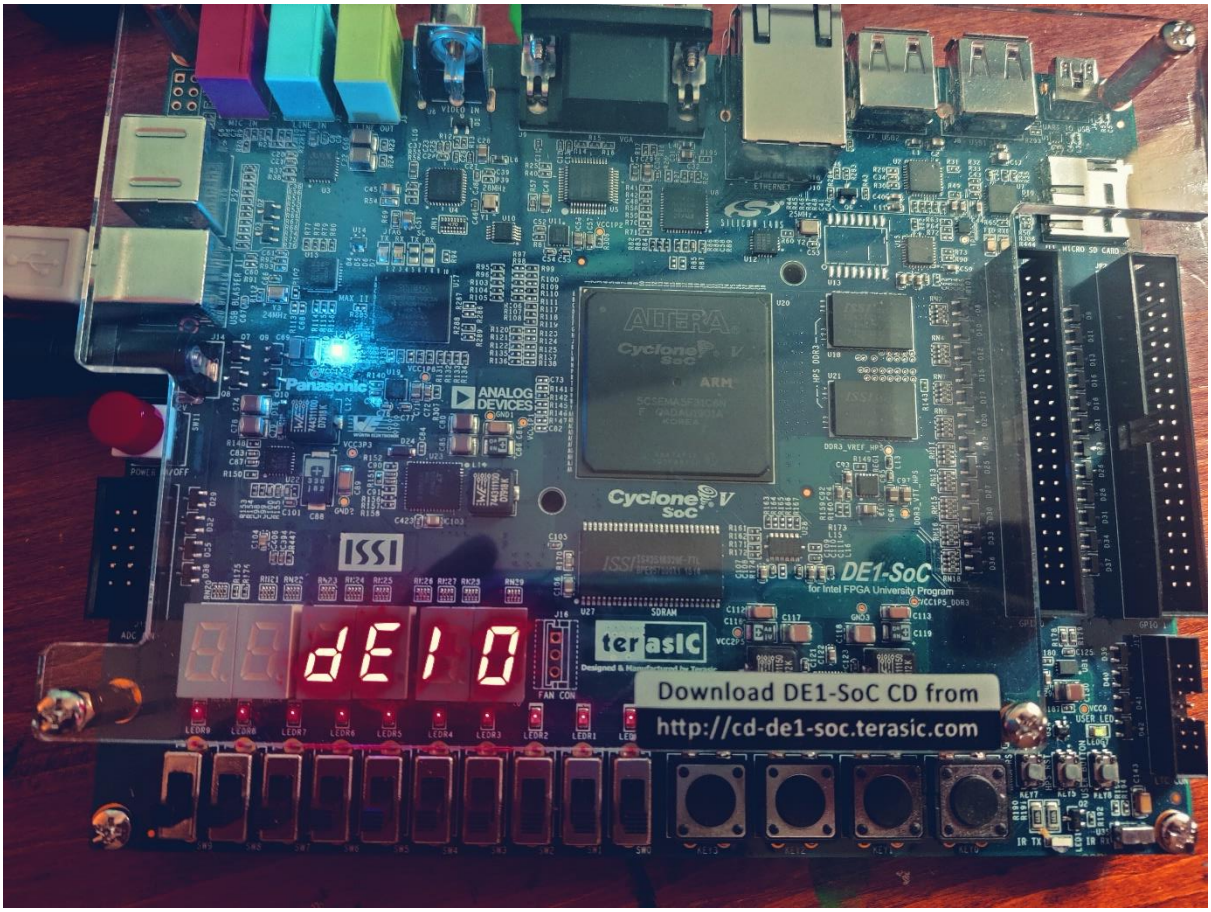


Figure 5.5: Sample Output

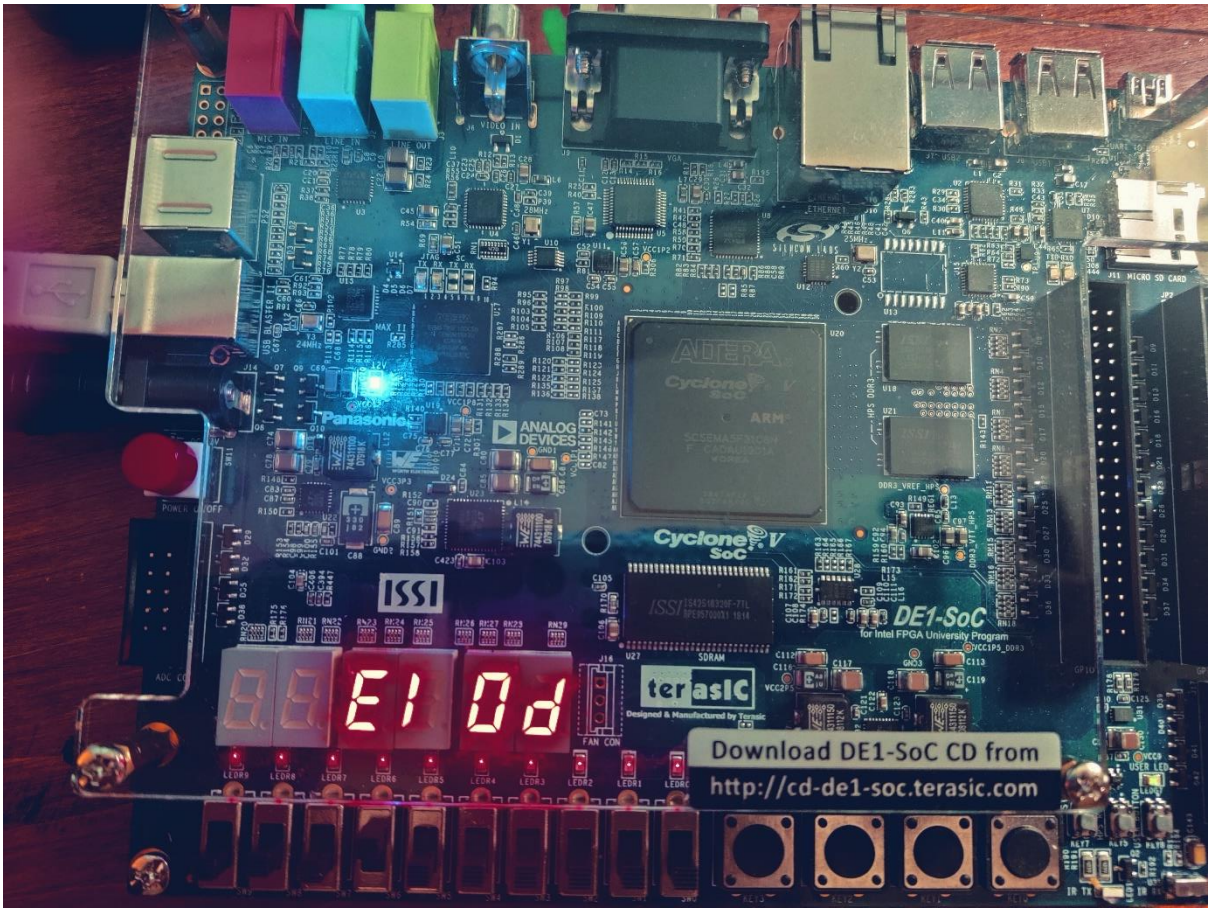


Figure 5.6: Sample Output (2)

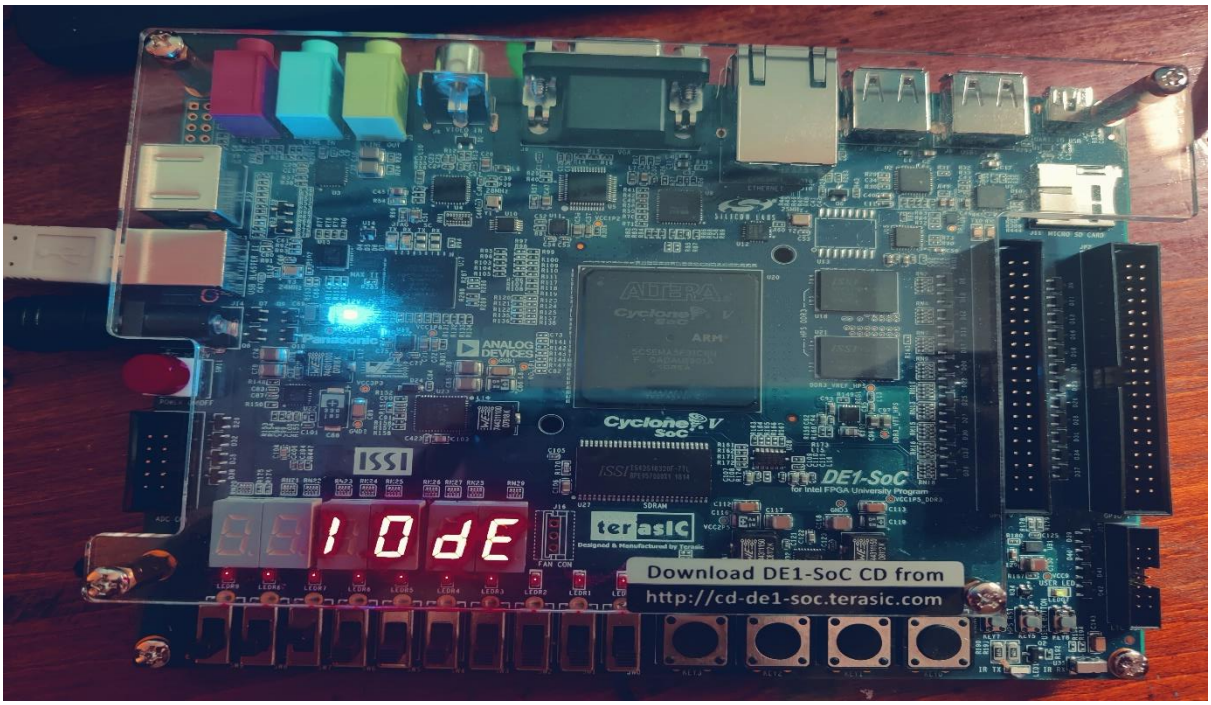


Figure 5.7: Sample Output (3)

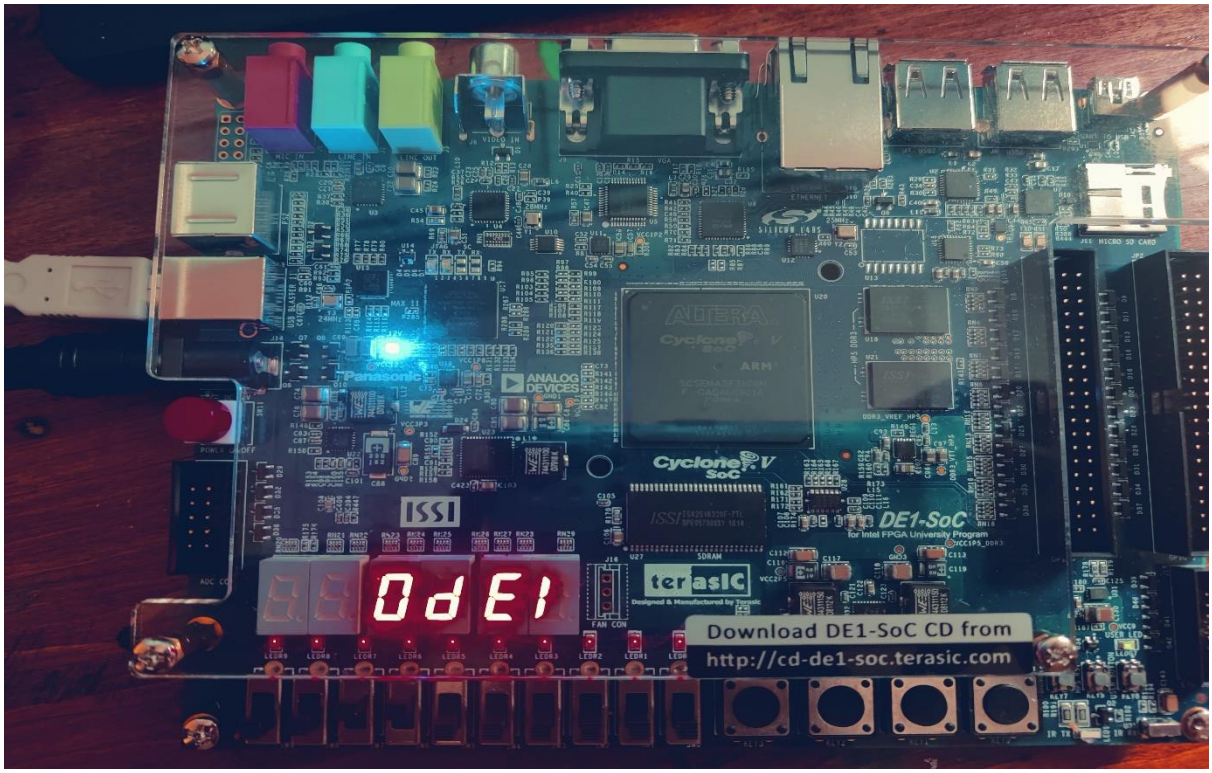


Figure 5.8: Sample Output (4)

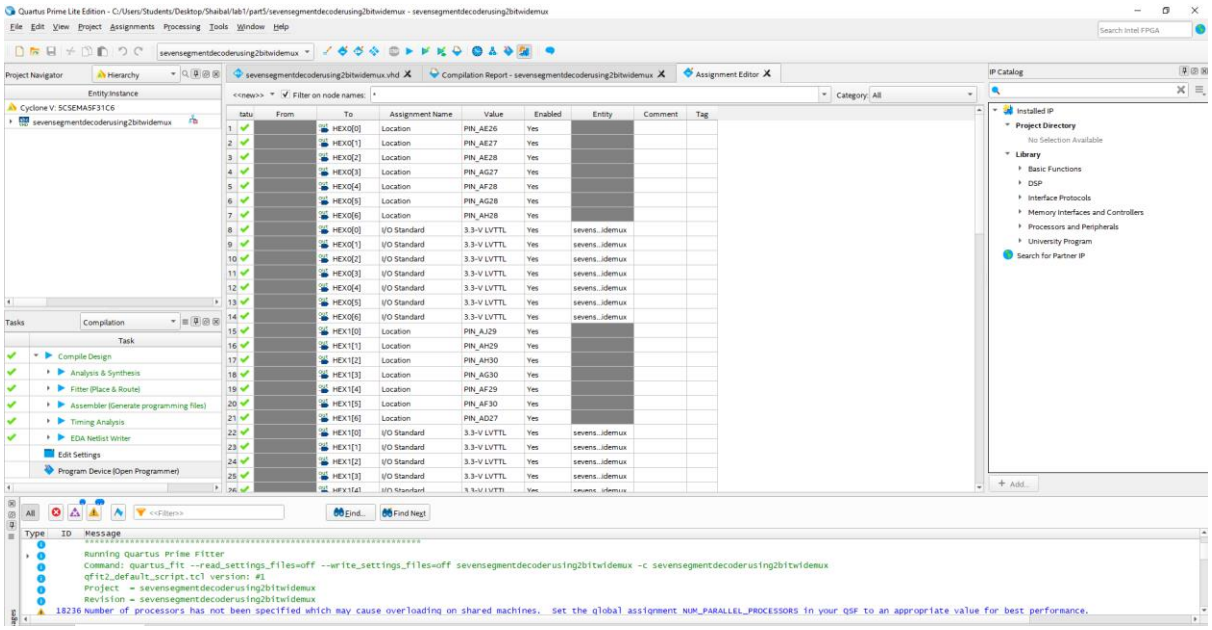


Figure 5.9: Pin assignment (1)

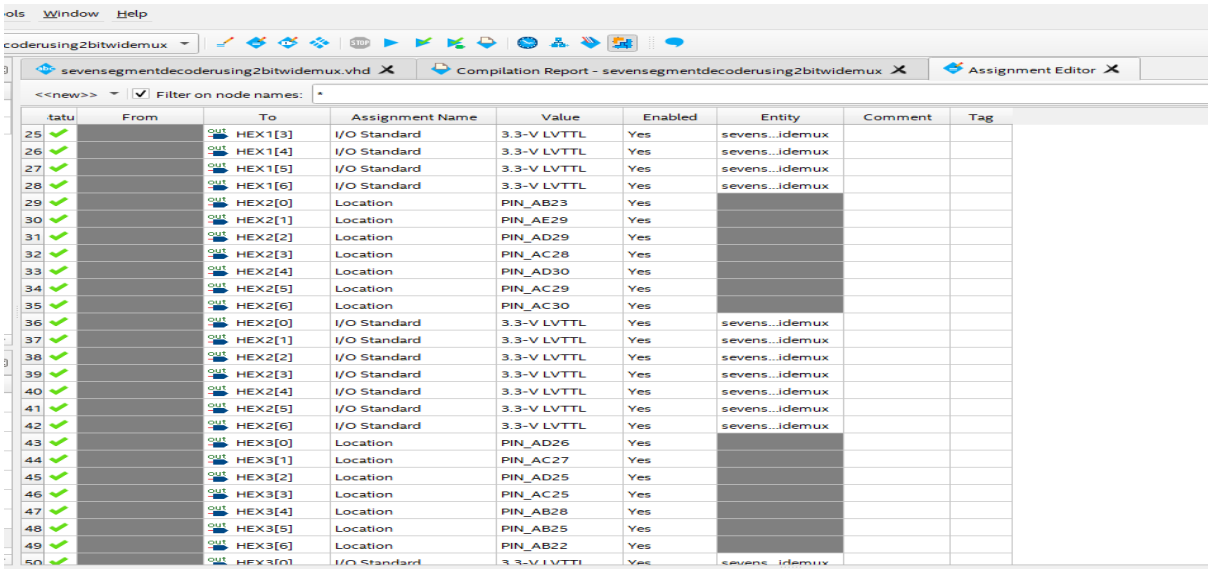


Figure 5.10: Pin assignment (2)

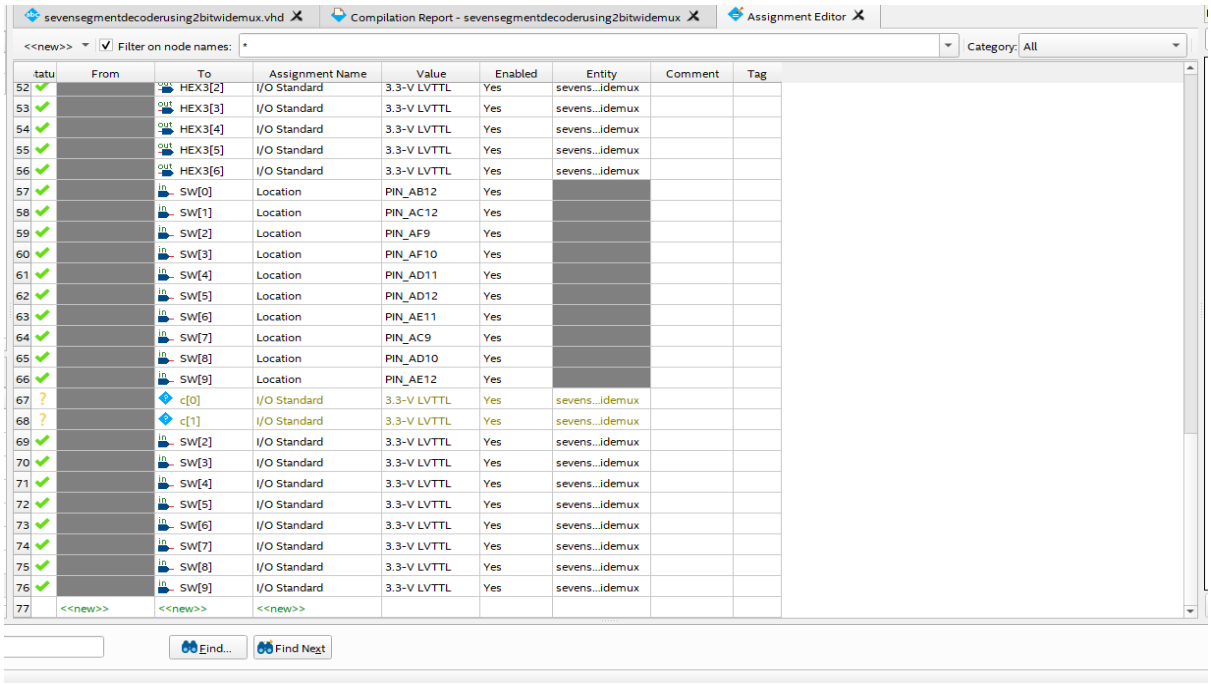


Figure 5.11: Pin assignment (3)

- **Part6:** The part6 is the extend version of part5. In the part 6, I need to view de10 with two blank 7-segment display. In desoc-1 board, we have 6 7-segment display. We use it all. I design a 6 to 1 mux for limitation of switches. Here I attach all the screenshot as well as sample output and the pin configuration below:

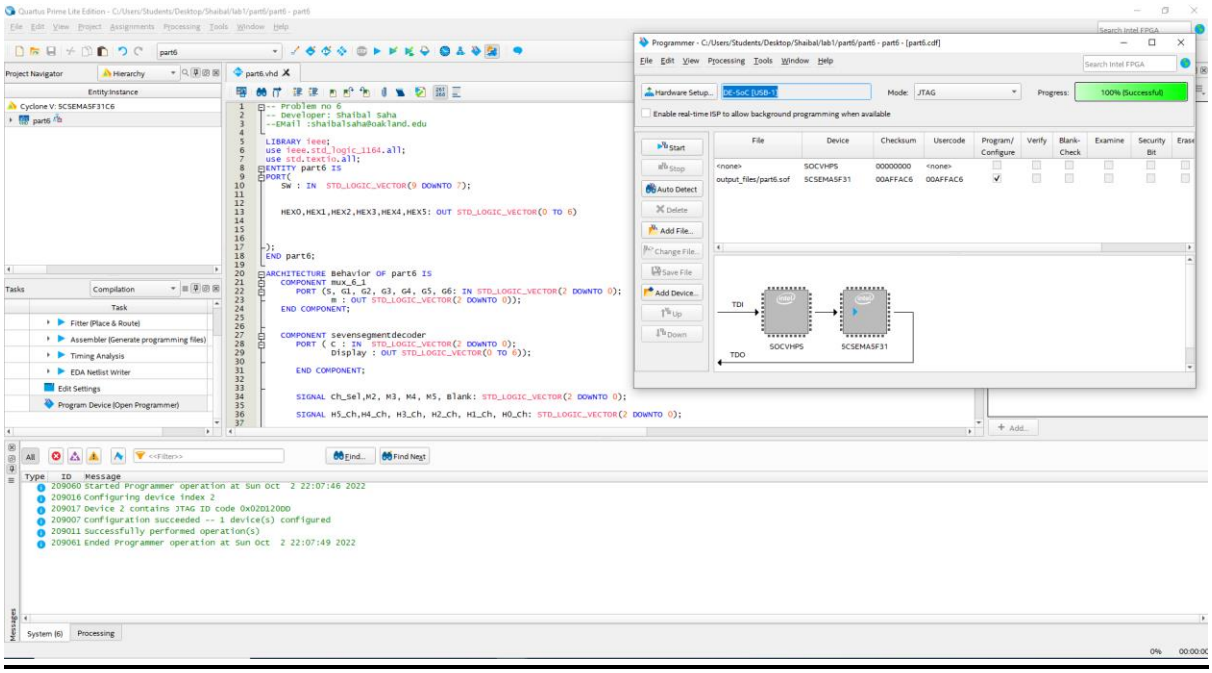


Figure 6.1: Code

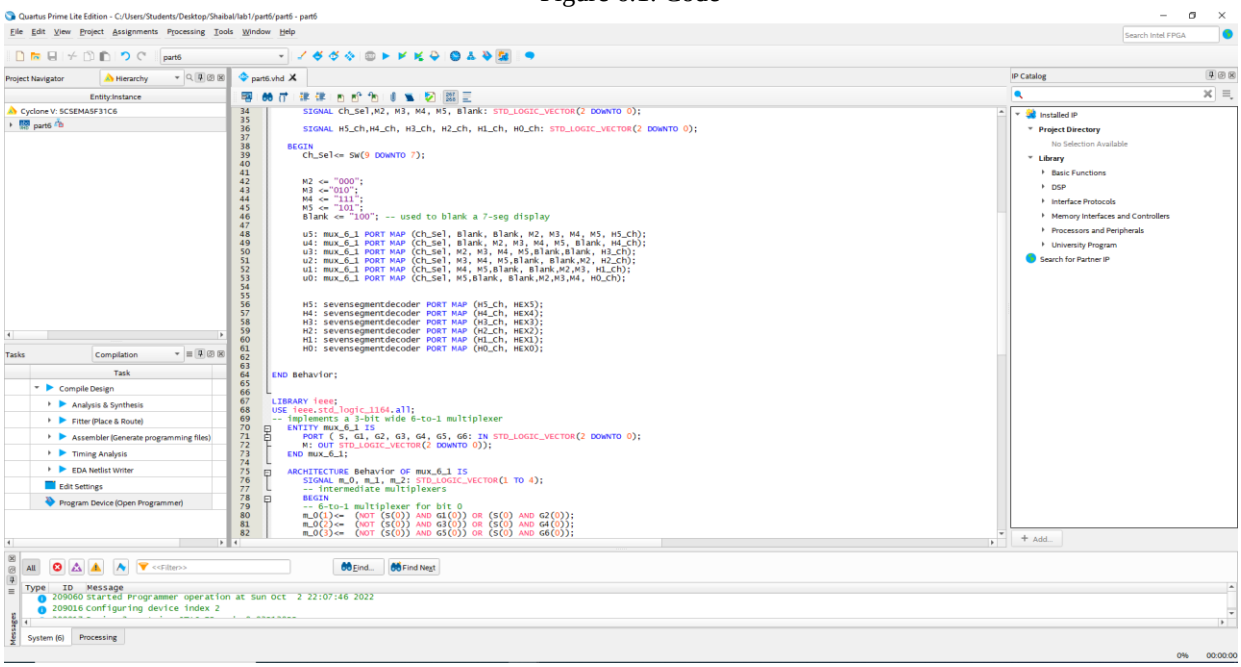


Figure 6.2: Code

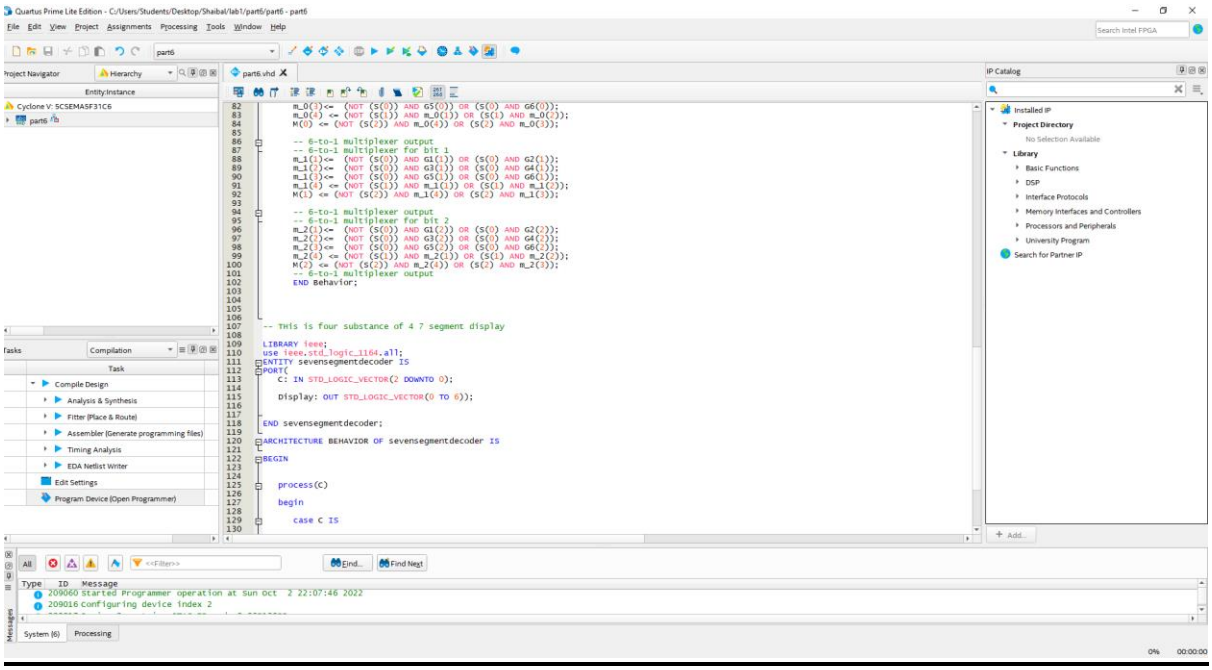


Figure 6.3: Code



Figure 6.4: Code

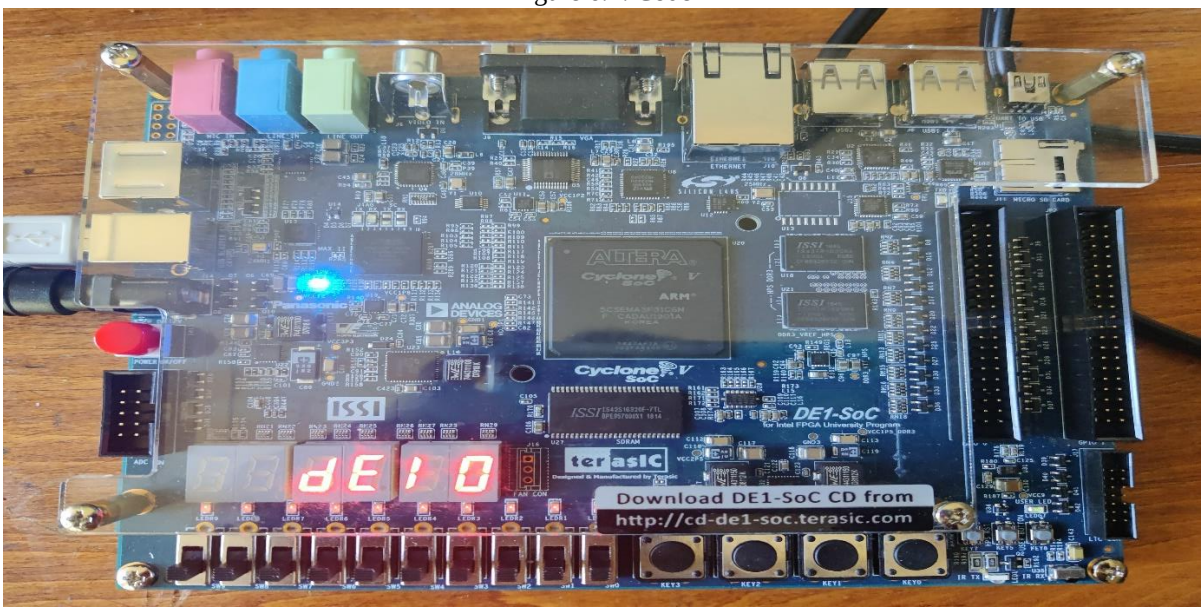


Figure 6.5: Sample Output

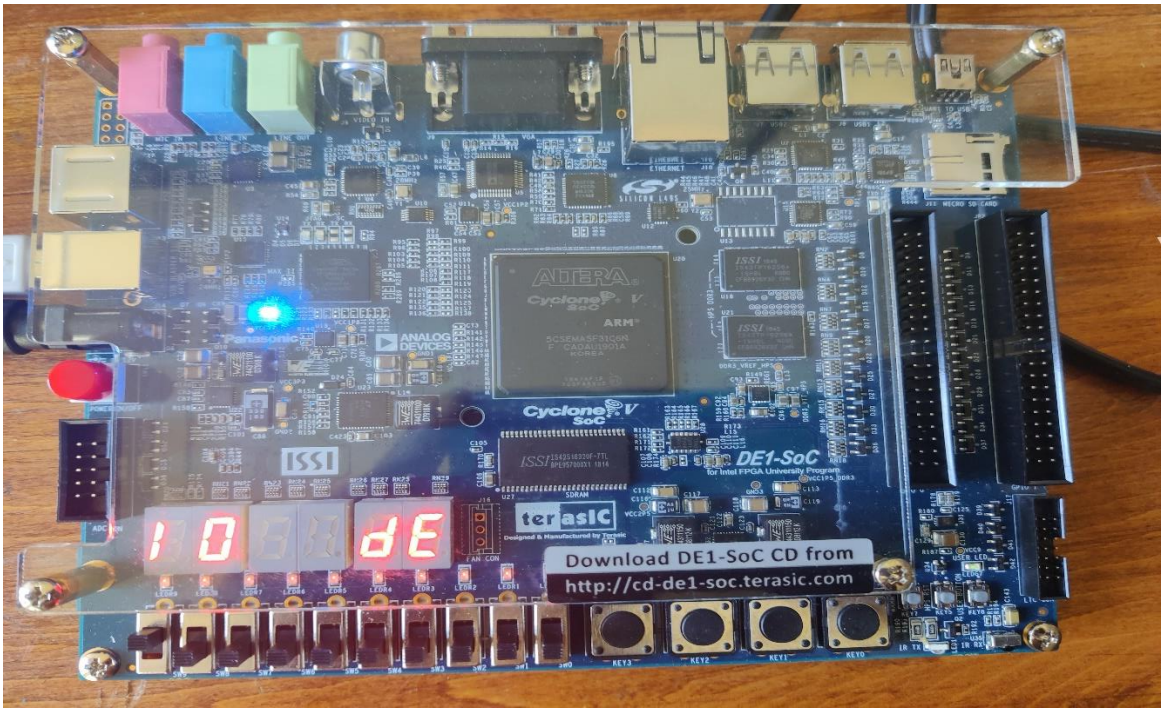


Figure 6.6: Sample Output

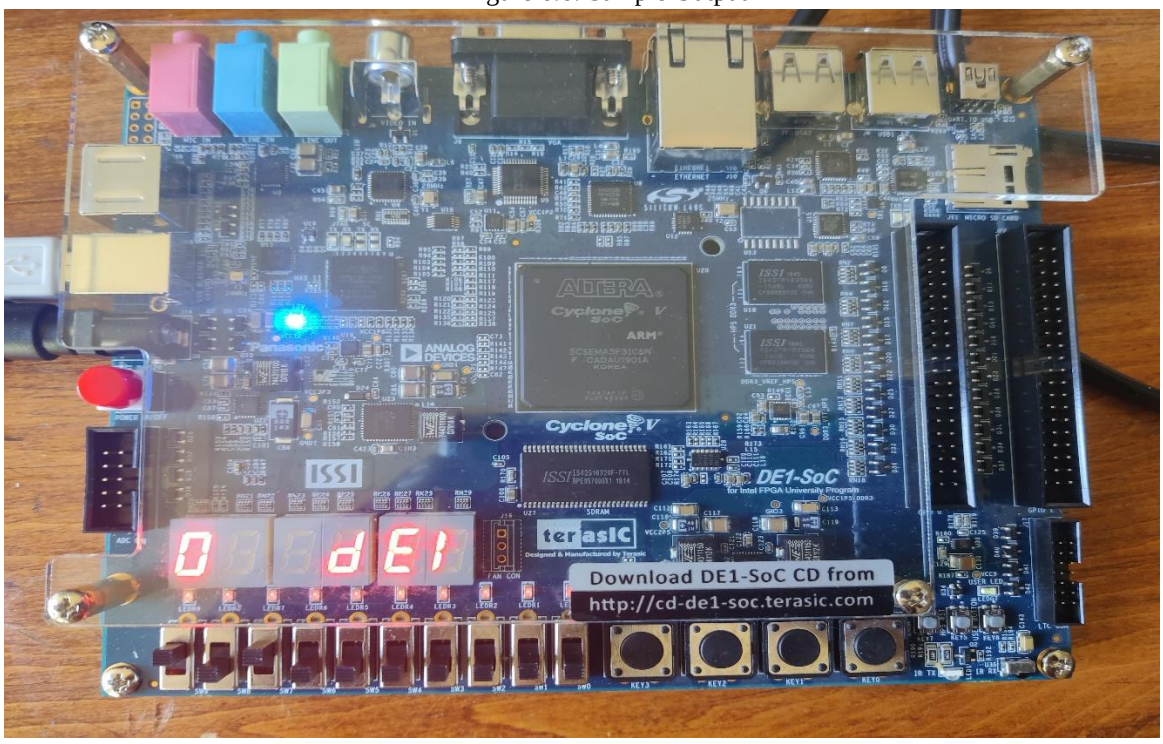


Figure 6.7: Sample Output

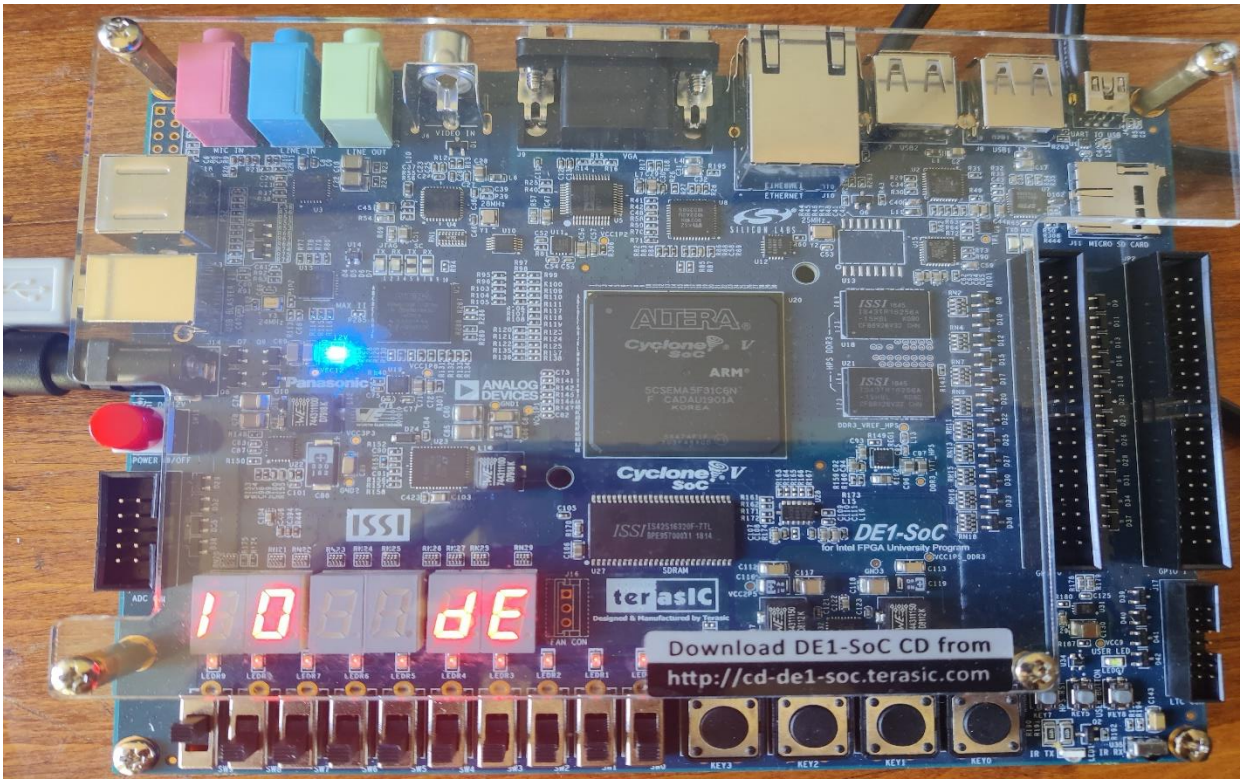


Figure 6.8: Sample Output

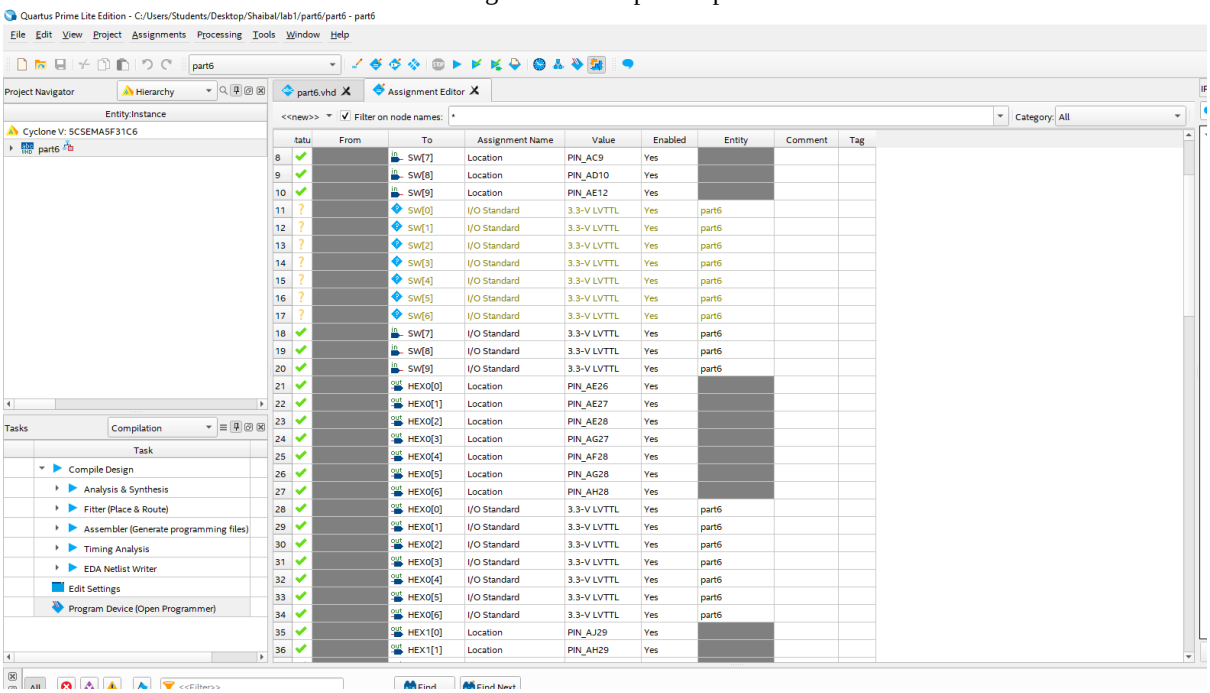


Figure 6.9: Pin assignment

part6.vhd | Assignment Editor

<<new>> | Filter on node names: *

	tatu	From	To	Assignment Name	Value	Enabled	Entity	Comment	Tag
50	✓		HEX2[1]	Location	PIN_AE29	Yes			
51	✓		HEX2[2]	Location	PIN_AD29	Yes			
52	✓		HEX2[3]	Location	PIN_AC28	Yes			
53	✓		HEX2[4]	Location	PIN_AD30	Yes			
54	✓		HEX2[5]	Location	PIN_AC29	Yes			
55	✓		HEX2[6]	Location	PIN_AC30	Yes			
56	✓		HEX2[0]	I/O Standard	3.3-V LVTTL	Yes	part6		
57	✓		HEX2[1]	I/O Standard	3.3-V LVTTL	Yes	part6		
58	✓		HEX2[2]	I/O Standard	3.3-V LVTTL	Yes	part6		
59	✓		HEX2[3]	I/O Standard	3.3-V LVTTL	Yes	part6		
60	✓		HEX2[4]	I/O Standard	3.3-V LVTTL	Yes	part6		
61	✓		HEX2[5]	I/O Standard	3.3-V LVTTL	Yes	part6		
62	✓		HEX2[6]	I/O Standard	3.3-V LVTTL	Yes	part6		
63	✓		HEX3[0]	Location	PIN_AD26	Yes			
64	✓		HEX3[1]	Location	PIN_AC27	Yes			
65	✓		HEX3[2]	Location	PIN_AD25	Yes			
66	✓		HEX3[3]	Location	PIN_AC25	Yes			
67	✓		HEX3[4]	Location	PIN_AB28	Yes			
68	✓		HEX3[5]	Location	PIN_AB25	Yes			
69	✓		HEX3[6]	Location	PIN_AB22	Yes			
70	✓		HEX3[0]	I/O Standard	3.3-V LVTTL	Yes	part6		
71	✓		HEX3[1]	I/O Standard	3.3-V LVTTL	Yes	part6		
72	✓		HEX3[2]	I/O Standard	3.3-V LVTTL	Yes	part6		
73	✓		HEX3[3]	I/O Standard	3.3-V LVTTL	Yes	part6		
74	✓		HEX3[4]	I/O Standard	3.3-V LVTTL	Yes	part6		
75	✓		HEX3[5]	I/O Standard	3.3-V LVTTL	Yes	part6		
76	✓		HEX3[6]	I/O Standard	3.3-V LVTTL	Yes	part6		
77	✓		HEX4[0]	Location	PIN_AA24	Yes			
78	✓		HEX4[1]	Location	PIN_Y23	Yes			

Figure 6.10: Pin Assignment

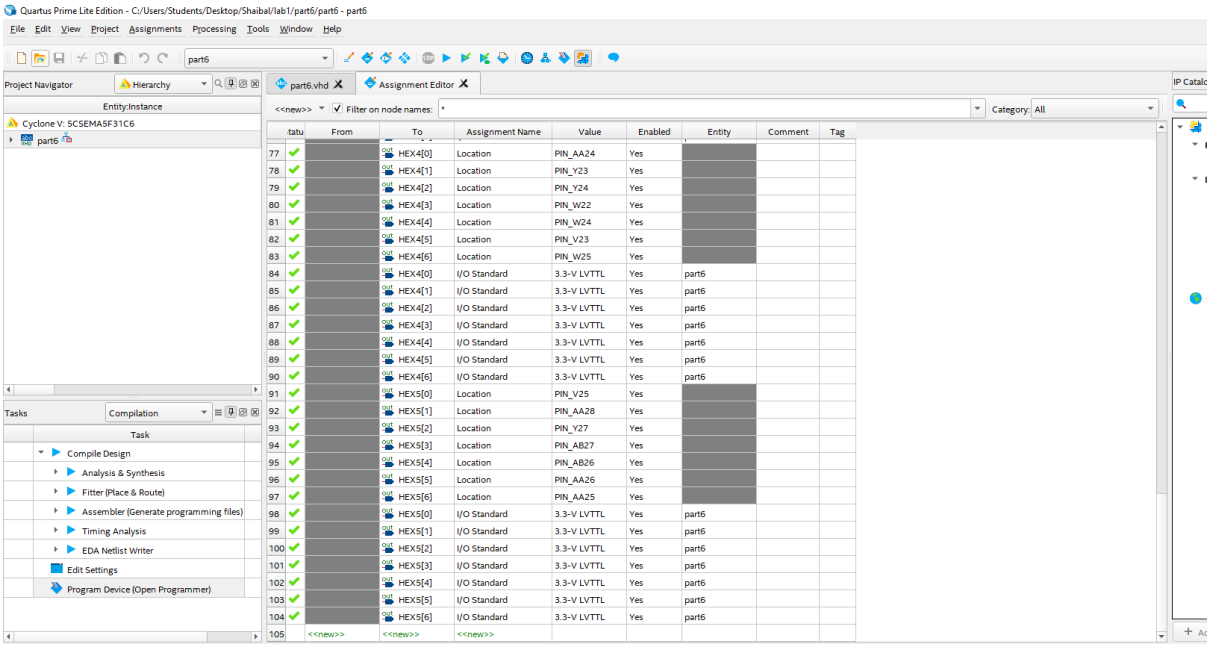


Figure 6.11: Pin Assignment

Note: All the source codes are inside lab folder.